

SwipeLab



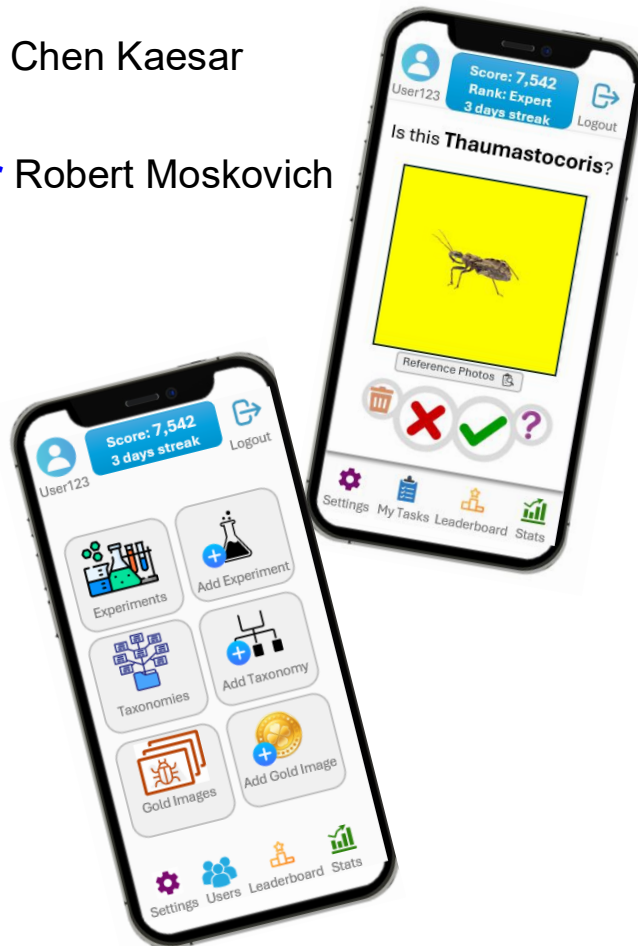
Application Requirements Document

Technical Advisor Chen Kaesar

Academic Advisor Robert Moskovich

Team Members

- Sagi Evroni
- Eden Bar
- Ofri Hanochi
- Avihoo Amos



SwipeLab	1
1. Introduction	3
1.1 The Problem Domain	3
1.2 Context	5
1.3 Vision	5
1.4 Stakeholders	6
1.5 Software Context	6
2. Usage Scenarios	11
2.1 User Profiles — The Actors	11
2.2 Use-Cases	13
2.3 Special Usage Considerations	27
3. Functional Requirements	29
3.1 Mobile Application Functionality	29
3.2 Backend API Server Functionality	32
3.3 Research Dashboard Functionality	34
3.4 Administrative Functionality	36
4. Non-Functional Requirements	37
4.1 Implementation Constraints	37
4.2 Platform Constraints	39
4.2.1 SE Project Constraints	40
4.3 Special restrictions & limitations	40
5. Risk assessment & Plan for the proof of concept	41
Appendices	46

1.Introduction

SwipeLab is a mobile application designed to assist researchers in collecting and classifying visual data through crowd participation. Users receive a series of reference images and classify them by swiping – similar to a Tinder-like interface (Yes / No / Maybe). The labeled data is used to train and improve research models, providing an engaging, user-friendly experience for contributors.

1.1 The Problem Domain

SwipeLab operates in the domain of **ecological and agricultural research based on insect monitoring systems**.

Across Israel (and globally), agricultural and ecological researchers deploy **insect traps** in dozens of field locations. These traps collect hundreds to thousands of insects and are periodically scanned into high-resolution images.

The collected images are stored in large research databases (e.g., [STARdbi](#)), forming datasets that include: Hundreds of thousands of insect detections, Multiple geographic locations, Longitudinal data over time.

These datasets are used to monitor: Invasive species spread, Biological control agents, Disease-carrying insects, Population dynamics, etc.

The ultimate goal is to enable **early detection and data-driven intervention** in agricultural threats and researches.

The Core Scientific Workflow

The research workflow includes:

1. **Trap Deployment** in ecological/agricultural sites

2. **Image Scanning** of traps

3. **Object Detection Models** identifying insect candidates

4. **Taxonomic Classification** (species / genus / family)

6. **Model Training & Ecological Analysis**



However, the classification stage requires **human-labeled data**.

Each detected insect image must be manually classified by experts according to taxonomic criteria.

The Core Domain Problem: The Labeling Bottleneck

The domain **suffers from a structural bottleneck**:

- Object detection models generate large volumes of insect candidate images.
- Each image must be labeled manually or automatically.
- Only a small number of trained experts can perform reliable taxonomic classification.
- Data collection rate exceeds labeling capacity.

As a result:

- Large portions of collected data remain unused.
- Model training is delayed.
- Research insights are slowed.
- Time-sensitive ecological threats may spread before action can be taken.



This mismatch between **data generation rate** and **expert labeling capacity** is the central domain problem.

SwipeLab in the Problem Domain

SwipeLab is designed specifically to address the key bottlenecks in insect-trap data labeling. Its **main focus** is to make large-scale, expert-quality classification feasible by non-experts.

SwipeLab sits at the intersection of ecological research, machine learning, and citizen science. It is **not just a labeling tool**, but a **scalable human-in-the-loop platform** that converts otherwise unusable datasets into actionable research resources.

Impact in the Domain:

By bridging the gap between high-volume data collection and expert classification, SwipeLab enables:

- Faster creation of training data for detection and classification models.

- More comprehensive ecological analysis across multiple locations.
- Early detection and monitoring of agricultural threats like invasive pests.

In short, SwipeLab **transforms the insect-classification workflow**, turning a slow, expert-limited process into a scalable, volunteer-driven, gamified system without compromising data quality.

1.2 Context

This project is part of the larger [STARdbi ecosystem](#), a comprehensive database and research infrastructure containing:

- 1,740+ trap scan images
- 52 collection sites
- ~350,000 individual insects
- Several active research projects requiring fast and accurate labeled data



The system interacts with several external components:

- **The STARdbi Dataset API**
Provides batches of trap scan images and metadata. Acts as the authoritative source of raw data.
- **SwipeLab Application (Our System)**
A mobile app through which non-experts perform binary classification tasks using swipe gestures. Converts unlabeled images into structured labeled data.
- **SwipeLab Backend Server**
Manages authentication, user profiles, scoring, data validation, and communication with the dataset API. Ensures secure and efficient data transfer.
- **Researcher Tools / Model Training Pipelines**
Consume the labeled data produced by SwipeLab for training and improving object detection and classification models.

1.3 Vision

The vision of the SwipeLab project is to **accelerate insect image labeling at scale** by transforming a scientific bottleneck into an accessible, engaging, and gamified user experience. The project aims to:

- Enable **citizen science participation** by simplifying classification into intuitive swipe-based binary decisions.
- Significantly **increase the volume and speed** of dataset labeling for STARdbi.
- Improve the performance of **machine learning models** used in ecological, agricultural, and public-health research.
- Ensure **data quality** through consistency checks, credibility scoring, and expert comparison.
- Create a system that is **easy to use, mobile-friendly, and secure**, encouraging users to contribute during their free time.

Ultimately, the system strives to support ongoing scientific projects by providing a scalable, cost-effective alternative to expert labeling—empowering non-experts to meaningfully contribute to scientific progress.

1.4 Stakeholders

- **1.4.1 Researchers (Customers)**
- This group is the primary customer for the project, represented by Associate Professor Chen Keasar. They are researchers working on the **STARdbi project**, a large-scale database of insect trap scans containing approximately 350,000 individual insects. Their main challenge is the "labeling bottleneck," as their machine learning models are "data-hungry" and require vast amounts of accurately labeled data. Their vision is to use this "Tinder for insects" app to enable "citizen science", capturing high-quality labeled data at scale. They require a research dashboard and robust quality control, including the use of "gold" (test) images to measure user accuracy.
- **1.4.2 Volunteers & Students (End Users)**
- This group consists of non-expert users, such as students and volunteers, who will act as "citizen scientists". Their participation is crucial for scaling the labeling process, as they can contribute during "small pockets of free time". To combat low participation, their primary requirement is an accessible, quick, and engaging game-like experience. The app must feature a simple "Yes / No / I Don't Know" workflow optimized for swipe gestures. To keep this group motivated, the system will rely heavily on gamification elements like points, badges, leaderboards, and a personal statistics dashboard.

1.5 Software Context

System Architecture Overview

The SwipeLab system consists of three primary software components that work together to deliver the complete solution:

1.5.1 Mobile Application (SwipeLab App)

The mobile application serves as the primary interface for citizen scientists to engage with image classification tasks.

Major Inputs:

- User authentication credentials (via OAuth providers or email/password)
- Classification responses (swipe gestures: left for "No", right for "Yes", up for "I Don't Know")
- User profile information and preferences
- Device metadata (screen resolution, OS version, device ID)

Core Functionality:

- User authentication and profile management
- Image presentation with smooth, responsive swipe interface
- Real-time feedback on classification decisions
- Progress tracking and statistics visualization
- Gamification elements display (points, badges, achievements)
- Leaderboard viewing with filtering options
- Offline classification queue management
- Push notification handling for engagement reminders
- Tutorial and onboarding workflows

Processing:

- Local caching of images for smooth user experience
- Client-side validation of user inputs
- Synchronization of local state with backend server
- Animation rendering for swipe gestures and transitions
- Image pre-loading and memory management

Major Outputs:

- Classification submissions (image ID, user ID, classification decision, timestamp, response time)
- User engagement metrics (session duration, images classified per session)
- Error reports and crash logs
- User feedback and feature requests

Platforms: iOS (iPhone/iPad) and Android (smartphones/tablets)

1.5.2 Backend API Server

The backend server manages all business logic, data persistence, authentication, and coordination between mobile clients and research dashboards.

Major Inputs:

- Classification submissions from mobile clients
- Dataset uploads from researchers (image files, metadata, classification criteria)
- Authentication requests from users and researchers
- Quality control image configurations (gold standard labels)
- Research queries and data export requests
- User credibility feedback (from gold standard image comparisons)

Core Functionality:

- RESTful API endpoints for all client-server communication
- User authentication and authorization (JWT-based)
- Dataset management (upload, storage references, task assignment)
- Classification task distribution (intelligent assignment to users)
- Quality control enforcement (inserting gold standard images at intervals)
- Credibility scoring algorithms (tracking accuracy on gold images)
- Consensus calculation (aggregating multiple user responses)
- Data aggregation and analytics computation
- User reward calculation (points, badge eligibility)
- Leaderboard ranking computation
- Data export and formatting for ML pipelines
- Rate limiting and API abuse prevention
- Logging and monitoring

Processing:

- Weighted consensus algorithms considering user credibility scores
- Statistical analysis of inter-annotator agreement
- Dynamic task difficulty assignment based on user performance
- Fraud detection (identifying bot accounts or spam classifications)
- Performance optimization through caching and database indexing
- Asynchronous job processing for heavy computations

Major Outputs:

- API responses (JSON format) to mobile and web clients
- Labeled datasets with metadata (CSV, JSON formats)
- Analytics reports and statistics
- User credibility scores and rankings
- Audit logs and system health metrics
- Email notifications for researchers
- Webhook callbacks to external ML systems

Technologies: Java Spring Boot backend framework, PostgreSQL database, Redis cache, cloud storage integration (AWS S3 or similar)

1.5.3 Research Dashboard (Web+Mobile Application)

The research dashboard provides researchers with comprehensive tools to manage datasets, monitor progress, and analyze results.

Major Inputs:

- Researcher authentication credentials
- Dataset upload requests (bulk image files)
- Classification task configurations (question text, category definitions)
- Gold standard image labels (expert-verified correct answers)
- Quality threshold parameters (minimum consensus percentage, minimum credibility score)
- Export format preferences and filters

Core Functionality:

- Researcher account management
- Dataset upload and organization interface
- Classification task creation wizard
- Gold standard image management
- Real-time progress monitoring dashboards
- User performance analytics and visualization
- Consensus analysis tools
- Data quality assessment reports
- Labeled data export with filtering options
- User management (viewing, blocking problematic users)
- System configuration and parameter tuning

Processing:

- Large file upload handling with progress tracking
- Data visualization rendering (charts, graphs, heatmaps)
- Complex filtering and search operations
- Report generation in multiple formats
- Data export transformation and formatting

Major Outputs:

- Labeled datasets (downloadable in multiple formats: CSV, JSON, COCO format)
- Analytics visualizations (progress charts, consensus distributions, user engagement metrics)
- Quality reports (inter-annotator agreement, confidence scores)
- User activity logs and credibility breakdowns
- System status notifications

Platform: Web-based responsive application (accessible via modern browsers and mobile phones)

System Data Flows

Classification Workflow:

1. Researcher uploads image dataset and creates classification task via Research Dashboard
2. Backend stores images in cloud storage and creates task records in database
3. Mobile app user requests new classification task
4. Backend intelligently selects next image (mixing regular and gold standard images)
5. User views image and makes classification decision via swipe gesture
6. Mobile app submits classification to backend with metadata
7. Backend records classification, updates user statistics, and calculates rewards
8. If gold standard image, backend updates user credibility score
9. Backend aggregates multiple user responses to calculate consensus
10. Researcher views progress and analytics via Research Dashboard
11. When sufficient consensus achieved, researcher exports labeled dataset

Quality Control Loop:

1. Backend periodically inserts gold standard images (known correct answers) into user task queues
2. User unknowingly classifies gold standard image
3. Backend compares user's answer against expert label
4. Backend updates user's credibility score based on accuracy
5. Future classifications from this user are weighted by their credibility score
6. Low-credibility users receive more frequent gold standard images for validation
7. Consistently inaccurate users may be flagged or their contributions down-weighted

Example Use-Cases

Use Case 1: Non-Expert User Performs Labeling

1. User logs in.
2. System displays three expert-verified examples ("wasp / not wasp").
3. User swipes right/left to label new images.
4. System stores labels and updates the user's stats.
5. User progresses through weekly challenges and rewards.

Use Case 2: Expert Defines a New Task Type

1. Expert uploads a set of example images.

2. Defines the binary labeling target (e.g., “mosquito / not mosquito”).
3. System generates a batch of unlabeled sub-images.
4. Task becomes available for users.

Use Case 3: ML Researcher Exports Data

1. Researcher selects a time range or trap site.
2. System exports aggregated labels with consensus confidence.
3. Model training begins using these annotations.

Use Case 4: Anti-Bot Mitigation

1. System monitors suspicious behavior (rapid clicking, repeated patterns).
2. Flags the session or requests a “human verification swipe test.”
3. Bot-like responses are excluded from datasets.

2. Usage Scenarios

2.1 User Profiles — The Actors

Actors represent external entities that interact with the system and exchange data with it. They may be human or non-human, but in all cases, they exist outside the system boundary.

Primary Human Actors

1. Volunteer User – (Non-Expert / Expert User)

A general public user with no background in entomology or an expert researcher that participates in the labeling process.

Characteristics:

- Uses swipe-based interface to classify insect images.

- Participates for curiosity, contribution to science, competition, or gamification rewards.
- Expected to classify many images at variable accuracy.
- Uses web/mobile app.
- No expert knowledge required.

Interactions with the system:

- Receives training examples.
- Performs binary classification tasks.
- Earns points, badges, weekly rankings.
- May review personal performance stats.

2. Researcher Admin User (Entomologist / Researcher)

A trained professional with biological or ecological expertise.

Characteristics:

- Provides verified example images.
- Reviews labeling outputs and supervises overall label quality.
- Defines new labeling tasks (e.g., “Mosquito / Not Mosquito”).
- Analyzes aggregated labeling statistics.

Interactions with the system:

- Uploads or approves example images.
- Create Tasks for users to label.
- Reviews consensus labels and data coverage.
- Downloads aggregated datasets.
- Monitors quality metrics and system dashboards.

Non-Human Actors

3. STARdbi Database

External data repository containing insect trap scan images.

Interactions with the system:

- Provides sub-images for tasks.
 - Receives refined labels produced by SwipeLab.
-

2.2 Use-Cases

2.2.0 UC-0: Register New Account

- **Primary Actor:** Guest (Unauthenticated User)
- **Supporting Actor:** SwipeLab Backend
- **Description:** A new user creates a SwipeLab account using either username/password registration or Google authentication.
- **Trigger:** The user opens the application and selects "Sign Up."

- **Preconditions:**

1. The app is installed on the user's device.
2. The device has an active internet connection.
3. The user does not already have an account in SwipeLab.

- **Postconditions (Success):**

1. A new user account is created in the system.
2. The system issues an access token and refresh token.
3. A secure session is established.
4. The user profile is initialized (points = 0, credibility score initialized).
5. The user is redirected to the main task screen.

- **Postconditions (Failure):**

1. No account is created.
2. The system displays an appropriate error message.

- **Main Success Scenario (Username & Password Registration):**

1. The user opens the application.
2. The system displays the Login / Registration screen.

3. The user selects "Create Account."
4. The user enters:
 - Username
 - Email address
 - Password
5. The system validates:
 - Username uniqueness
 - Email uniqueness
 - Password strength
6. The system securely stores the user credentials (hashed password).
7. The backend creates a new user record.
8. The backend generates authentication tokens.
9. The system logs the user in automatically.
10. The user is redirected to the main classification screen.

● **Main Success Scenario (Google Registration):**

1. The user taps "Sign Up with Google."
2. The system initiates OAuth2 authentication.
3. Google returns a valid ID token.
4. The backend verifies the token.
5. If no existing user is linked to this Google account:
 - A new user profile is created.
6. The backend issues authentication tokens.
7. The user is redirected to the main task screen.

● **Alternative Flows:**

○ **5a. Username Already Exists**

1. The system detects duplicate username.
2. The system displays: "Username already taken."
3. Registration is halted.

○ **5b. Email Already Exists**

1. The system detects duplicate email.
2. The system displays: "Email already registered."

○ **5c. Weak Password**

1. The password does not meet security policy.
2. The system displays password requirements.

○ **3a (Google). Invalid or Expired Token**

1. System rejects token.
2. System prompts user to retry authentication.

2.2.1 UC-1: Login (Authenticate and Onboard)

- **Primary Actor:** Volunteer (End User)
 - **Description:** The user securely logs into the application and is shown a brief introduction to the project's purpose.
 - **Trigger:** The user opens the application for the first time or after being logged out.
 - **Preconditions:**
 1. The app is installed on the user's mobile device.
 2. The device has an active internet connection.
 - **Postconditions:**
 1. The user is authenticated, and a secure session is created.
 2. The user's profile (for tracking points and credibility) is created or retrieved.
 - **Main Success Scenario:**
 1. The user opens the app and is presented with the "Login / Onboarding" screen.
 2. The screen provides a short intro explaining the project's purpose and privacy.
 3. The user taps the "Google Sign-In" button.
 4. The system uses OAuth2 to securely authenticate the user with their Google account.
 5. The system verifies the user's token and links them to their SwipeLab profile.
 6. The system proceeds to the main "Classification Screen" (UC-2) for regular-user & proceeds to the main "Management Dashboard" () for researcher-admin.
-

2.2.2 UC-2: Classify Image Stream

- **Primary Actor:** Volunteer (End User)
- **Description:** The user classifies a stream of images using simple swipe gestures to earn points and contribute data to the research project.
- **Trigger:** The user is logged in and is on the "Classification Screen".
- **Preconditions:**
 - The user is authenticated (UC-1).
 - The external StarDBI dataset service is reachable / There are cached unclassified images already stored in the user's device.
- **Postconditions:**
 - The user's classifications are stored in the system.
 - The task status is ACTIVE.
 - The user's points and reliability score are updated.
- **Main Success Scenario:**
 - The system retrieves and displays an image from the STARdbi dataset API.
 - The user analyzes the image and swipes (or taps) "Yes," "No," "I Don't Know" or "Trash".
 - The system provides immediate visual feedback for the action.
 - The system securely records the user's label and associated metadata (e.g., response time).
 - The system updates the user's points and progress bar.
 - The system serves the next image in the batch.

- **Alternative Flows:**
 - **2a. System serves a "Trap Image":**
 1. The system retrieves and displays a "gold-standard" image with a known label.
 2. The user provides a classification.
 3. The system compares the user's answer to the known label and updates the user's hidden "credibility score".
-

2.2.3 UC-3: View Task Details

- **Primary Actor:** Volunteer (End User)
 - **Description:** The user views detailed information about a specific classification task.
 - **Trigger:** The user selects a task from his assigned tasks list / public tasks.
 - **Preconditions:**
 1. The user is authenticated (UC-1).
 2. The task is ACTIVE.
 - **Postconditions:**
 1. Task details are displayed.
 - **Main Success Scenario:**
 1. The system retrieves:
 - Target species
 - Reference images
 - Task description
 - Deadline
 - Progress statistics
 2. The system displays the detailed task screen.
-

2.2.4 UC-4: Play Task

- **Primary Actor:** Volunteer (End User)
- **Supporting Actors:** StarDBI
- **Description:** The user selects a task and initiates gameplay. From this point forward, all image batches retrieved for classification are restricted to the selected task's dataset obtained from StarDBI.
- **Trigger:** The user presses the "Play" button on a selected task.
- **Preconditions:**
 1. The user is authenticated.
 2. The selected task exists in the system.
 3. The selected task status is **Active**.
 4. Internet connectivity is available.
- **Postconditions (Success):**
 1. The selected task is set as the user's current active task.
 2. The system associates the task ID with the user session.
 3. The first batch of images retrieved belongs exclusively to the selected task dataset.
 4. The user is redirected to the classification screen.
- **Postconditions (Failure):**
 1. No task session is started.
 2. The user remains on the task selection screen.
 3. An error message is displayed.

- **Main Success Scenario:**

1. The user navigates to the Tasks screen.
2. The system displays a list of active tasks.
3. The user selects a task.
4. The system displays the task details.
5. The user presses "Play."
6. The system validates:
 - Task exists.
 - Task status is Active.
7. The system sets the task as the user's active task session.
8. The system requests a filtered batch of images corresponding to the selected task from StarDBI.
9. The external source returns task-specific images.
10. The system displays the first image for classification.

- **Alternative Flows:**

- 7a. Task Not Active**

1. System detects task status is not Active.
2. System returns error: "Task unavailable."
3. The session is not started.

- 9a. Image Retrieval Failure**

1. System fails to retrieve images from StarDBI.
 2. System retries according to policy.
 3. If failure persists:
 - The system displays: "Unable to retrieve images. Please try again later."
 - No session is started.
-

2.2.5 UC-5: View Personal Progress

- **Primary Actor:** Volunteer (End User)
- **Description:** The user accesses their personal statistics and the community leaderboard to track their progress and ranking, which directly supports the project goal of increasing user engagement.
- **Trigger:** The user taps on the "Progress" or "Leaderboard" tab/button in the app.
- **Preconditions:**
 1. The user is logged in (UC-1).
- **Postconditions:**
 1. None (this is a view-only action).
- **Main Success Scenario (Personal Stats):**
 1. User navigates to the "Progress Screen".
 2. The system retrieves and displays the user's personal statistics, including:
 - Total points earned.
 - Badges unlocked.
 - Rank and level.
 - Total classifications.
 - Accuracy vs. expert consensus.

- Time spent and streak days.
-

2.2.6 UC-6: View Leaderboard

- **Primary Actor:** Volunteer (End User)
 - **Description:** The user views community rankings to encourage engagement.
 - **Trigger:** The user taps the "Leaderboard" tab.
 - **Preconditions:** The user is authenticated (UC-1).
 - **Postconditions:** None.
 - **Main Success Scenario:**
 1. The system retrieves leaderboard data.
 2. The system displays: Monthly ranking ,All time ranking ,Current user rank
-

2.2.7 UC-7: Add New Classification Task

Primary Actor: Research Scientist (Administrator)

Description: The researcher creates a new classification task for a specific bug or organism type by uploading reference pictures and defining the classification question to guide mobile app users.

Trigger: The researcher navigates to the Research Dashboard and clicks "Create New Classification Task."

Preconditions:

1. The researcher has administrator role.
2. The researcher has prepared reference images showing the target organism.
3. At least one dataset exists or is being uploaded concurrently.

Postconditions:

1. A new classification task is created with reference pictures.
2. The task is associated with a dataset and available to mobile users.
3. Users can view reference pictures when classifying this task.

Main Success Scenario:

1. The researcher logs into the research dashboard.
2. The researcher clicks "Create New Classification Task."
3. The system displays the task creation form.
4. The researcher enters task details:
 - Task name (e.g., "Asian Giant Hornet Identification")

- Classification question (e.g., "Is this an Asian Giant Hornet?")
 - Brief description of identifying features
5. The researcher selects experiments (at least one) from STARdbi (these are the source dataset images).
 6. The researcher uploads 3-10 reference pictures showing clear examples of the target organism.
 7. For each reference picture, the researcher adds captions explaining key features (optional).
 8. The researcher configures quality parameters (minimum classifications per image, consensus threshold).
 9. The researcher reviews the configuration and clicks "Create Task."
 10. The system validates the task configuration and stores the reference images.
 11. The researcher associates the task with an existing dataset.
 12. The system activates the task and makes it available to mobile app users.

Alternative Flows:

- **5a.** No reference images uploaded: System warns but allows task creation if researcher confirms.
- **10a.** Validation fails (missing required information): System highlights issues and prevents creation until resolved.
- **11a.** No suitable dataset exists: System prompts researcher to upload dataset first.
- **13a.** User skips tutorial: Reference pictures remain accessible via info button during classification.

2.2.8 UC-8: Define Trap Pictures for User Reliability Grading

Primary Actor: Research Scientist (Administrator)

Description: The researcher defines "trap pictures" (also known as gold standard or quality control images) with known correct classifications. These images are inserted into user classification queues to measure accuracy and calculate user reliability scores.

Trigger: The researcher navigates to the SwipeLab "Research Dashboard" and selects an existing classification task to add quality control images.

Preconditions:

1. The researcher has administrator-level credentials.
2. A classification task has been created (UC-7).
3. The researcher has expert-verified correct labels for selected images.
4. At least 10 images are available in the dataset to serve as trap pictures.

Postconditions:

1. Trap pictures are designated in the system with their correct classifications.
2. The system begins inserting trap pictures into user classification queues at the specified frequency.

3. User reliability scores can be calculated based on accuracy on trap pictures.
4. The researcher can monitor user performance against trap pictures in real-time.

Main Success Scenario:

1. The researcher logs into the research dashboard.
 2. The researcher navigates to the classification task detail page.
 3. The researcher clicks on the "Quality Control" or "Trap Pictures" tab.
 4. The system displays all images in the dataset with their current classification status.
 5. The researcher selects images to designate as trap pictures (recommended: 5-15% of total dataset).
 6. For each selected trap picture, the researcher provides the correct classification (e.g., "Yes" - this is a bee, "No" - this is not a bee)
 7. The researcher configures trap picture parameters:
 - Insertion frequency (e.g., 1 trap picture every 10-15 regular images)
 - Whether to provide immediate feedback to users after trap picture classification
 - Reliability score impact weight (how much each correct/incorrect answer affects user score)
 - Minimum number of trap pictures a user must encounter before reliability score is calculated
 8. The researcher reviews the trap picture configuration and distribution settings.
 9. The researcher clicks "Activate Trap Pictures."
 10. The system validates that trap pictures represent diverse difficulty levels and image types.
 11. The system saves the trap picture configuration and begins inserting them into active user queues.
 12. The system displays a confirmation message: "Trap pictures activated. User reliability tracking is now enabled."
 13. The researcher can view real-time reliability statistics on the dashboard.
-

2.2.9 UC-9: View Task Analytics

- **Primary Actor:** Research Scientist (Administrator)
- **Description:** The researcher views analytical metrics and performance insights for a specific task based on collected classification data.
- **Trigger:** The researcher selects a task and navigates to the Analytics section.

- **Preconditions:**

1. The researcher is authenticated with administrator privileges.
2. The selected task exists in the system.
3. The task contains classification data.
4. Internet connectivity is available.

- **Postconditions (Success):**

1. Task analytics are retrieved from the system.
2. All requested analytical metrics are displayed.
3. No modification is made to task data.

- **Postconditions (Failure):**

1. No analytics are displayed.
2. An appropriate error message is shown.

- **Main Success Scenario:**

1. The researcher logs into the admin dashboard.
2. The system displays the list of tasks.
3. The researcher selects a task.
4. The researcher navigates to the “Analytics” section.
5. The system retrieves task analytics.
6. The system displays:
 - Completion percentage
 - Consensus rate
 - Species distribution analytics
 - Confusion matrix
 - Participation metrics
 - Credibility/quality metrics
 - Time-series analysis of classifications
7. The researcher reviews the analytics.

- **Alternative Flows:**

6a. No Classification Data Available

1. System displays: “No analytics available yet.”
2. The researcher remains on the analytics page.

5a. System Retrieval Failure

1. Analytics retrieval fails.
 2. System displays error message.
 3. Researcher may retry.
-

2.2.4 UC-10: Analyze and Export Data

- **Primary Actor:** Researcher (Administrator)
 - **Description:** The researcher logs into the web dashboard to monitor the quality of the incoming labels and export a cleaned dataset for use in model training.
 - **Trigger:** The researcher navigates to the SwipeLab "Research Dashboard" and logs in.
 - **Preconditions:**
 1. The researcher has administrator-level credentials.
 2. Volunteers have submitted a sufficient number of classifications.
 - **Postconditions:**
 1. The researcher receives a downloadable file (e.g., CSV, JSON) containing high-quality labeled data.
 - **Main Success Scenario:**
 1. The researcher logs into the research dashboard.
 2. The system displays an overview of statistics, user stats, and consensus levels.
 3. The researcher navigates to the "Export Data" section.
 4. The researcher applies filters, such as a minimum "user reliability score," to exclude low-quality data.
 5. The researcher clicks "Generate Export."
 6. The system queries the database, filters the labels, and generates a file in the selected format.
 7. The researcher downloads the file.
-

2.2.11 UC-11: Archive or Pause Task

- **Primary Actor:** Research Scientist (Administrator)
- **Description:** The researcher changes the status of a task to Pause or Archive, preventing further image classification within that task.
- **Trigger:** The researcher selects "Pause" or "Archive" on a task.
- **Preconditions:**
 1. The researcher is authenticated with administrator privileges.
 2. The task exists in the system.
 3. The task status is currently Active.
- **Postconditions (Success):**

1. The task status is updated to either Paused or Archived.
2. Users can no longer retrieve image batches for that task.
3. Active user sessions related to the task are invalidated.
4. The system logs the status change.

- **Postconditions (Failure):**

1. Task status remains unchanged.
2. An appropriate error message is displayed.

- **Main Success Scenario:**

1. The researcher logs into the admin dashboard.
2. The researcher selects a task.
3. The researcher clicks "Pause" or "Archive."
4. The system requests confirmation.
5. The researcher confirms the action.
6. The system updates task status.
7. The system confirms the update.
8. The task is no longer available for classification.

- **Alternative Flows:**

2a. Task Already Archived

1. System detects task status is already Archived.
2. System notifies researcher.
3. No change is made.

6a. Active Classification Sessions Exist

1. System detects users currently classifying images in the task.
2. System:
 - Either immediately terminates sessions, OR
 - Marks task unavailable for new batches while allowing current image to complete (based on system policy).
3. Task status is updated accordingly.

2.2.12 UC-12: Manage Recipients List

- **Primary Actor:** Research Scientist (Administrator)

- **Description:** The administrator manages the system recipients list by adding or removing recipient entities eligible for notifications, exports, or task-related communications.

- **Trigger:** The administrator navigates to the "Recipients Management" section.

- **Preconditions:**

1. The administrator is authenticated.
2. The administrator has management privileges.
3. Internet connectivity is available.

- **Postconditions (Success):**

1. Recipient list is updated (add/remove operation completed).
2. Changes are persisted in the backend database.
3. Audit log entry is created.

- **Postconditions (Failure):**

1. No changes are applied to the recipients list.
2. An appropriate error message is displayed.

- **Main Success Scenario (Add Recipient):**

1. The administrator navigates to "Recipients Management."
2. The system displays current recipients list.
3. The administrator selects "Add Recipient."
4. The administrator enters recipient details:
 - Name
 - Email address
 - Role/type (if applicable)
5. The system validates input format.
6. The system sends add-recipient request to backend.
7. The system verifies recipient does not already exist.
8. The system stores recipient record.
9. The system updates and displays the updated list.

- **Main Success Scenario (Remove Recipient):**

1. The administrator navigates to "Recipients Management."
2. The system displays current recipients list.
3. The administrator selects a recipient.
4. The administrator clicks "Remove."
5. The system requests confirmation.
6. The administrator confirms.
7. The system removes the recipient.
8. The system updates and displays the updated list.

- **Alternative Flows:**

- 7a. Recipient Already Exists**

1. System detects duplicate email.
2. System displays: "Recipient already exists."
3. No new record is created.

- 5a. Invalid Email Format**

1. System detects invalid email format.
2. System prompts administrator to correct input.

3. Flow returns to Step 4.
-

2.2.13 UC-13: Manage Users (Add / Block / Unblock)

- **Primary Actor:** Research Scientist (Administrator)
- **Description:** The administrator manages user accounts, including manually adding users (if supported), blocking users, or unblocking users.
- **Trigger:** The administrator navigates to the “User Management” section.

- **Preconditions:**

1. The administrator is authenticated.
2. Administrator privileges are verified.
3. The target user account exists (for block/unblock).
4. Internet connectivity is available.

- **Postconditions (Success):**

1. User status is updated accordingly (Active / Blocked).
2. Changes are stored in the swipelab system.
3. Audit log entry is created.
4. If blocked:
 - User can no longer log in.
 - Active sessions are invalidated.

- **Postconditions (Failure):**

1. No changes are applied to the user account.
2. An appropriate error message is displayed.

- **Main Success Scenario (Block User):**

1. The administrator navigates to “User Management.”
2. The system displays a list of users.
3. The administrator selects a user.
4. The administrator clicks “Block.”
5. The system requests confirmation.

6. The administrator confirms.
7. The system updates user status to "Blocked."
8. The system invalidates active tokens.
9. The system confirms the action success.

- **Main Success Scenario (Unblock User):**

1. The administrator selects a blocked user.
2. The administrator clicks "Unblock."
3. The system requests confirmation.
4. The administrator confirms.
5. The system updates user status to "Active."
6. The system confirms the update.

- **Main Success Scenario (Add User – If Supported by API):**

1. The administrator selects "Add User."
2. The administrator enters required user details.
3. The system validates input.
4. The backend creates a new user account.
5. The system confirms successful creation.

- **Alternative Flows:**

7a. User Already Blocked

1. System detects user status is already Blocked.
2. System notifies administrator.
3. No change is made.

7b. Attempt to Block Another Administrator

1. System detects target user has administrator privileges.
2. System denies operation.
3. Error message displayed: "Operation not permitted."

2.3 Special Usage Considerations

This section describes additional operational constraints and special circumstances influencing system usage.

1. High Variability of User Expertise

Users have no formal entomology training.
The system must:

- Provide intuitive examples
 - Avoid overwhelming users
 - Use simple, binary tasks only
-

2. Mobile-Focused Interaction

Many users will access the system via mobile devices.
Thus:

- UI must be swipe-optimized
 - Image resolution must be adjusted dynamically
 - Response-time heuristics must account for mobile delays
-

3. Data Sensitivity and Scientific Accuracy

The output feeds scientific research and ML model training.
Therefore:

- Label quality must be ensured through consensus
- Experts must be able to audit datasets

- Anti-bot detection must be robust
-

4. Large-Scale Data Operations

The STARdbi database contains:

- 1740 trap scans
- 350,000+ insect sub-images
- Multiple sites and trap types

The system must:

- Efficiently fetch and serve images
 - Support concurrent labeling by potentially many users
-

5. Motivation & Gamification

To maintain user engagement over long periods:

- System should include points, challenges, and leaderboards
 - Users should receive feedback and achievements
 - Weekly/monthly competitions must reset automatically
-

6. Accessibility & Inclusiveness

Users may have:

- Different devices
- Different abilities
- Limited time or attention

Therefore the system must:

- Be simple to use
 - Include readable UI elements
 - Avoid requiring scientific jargon
-

7. Reliability of User Data

Users may leave sessions mid-task, switch devices, or lose connectivity. System must:

- Save progress incrementally
- Support seamless reconnection
- Prevent partial data corruption

3. Functional Requirements

This chapter describes the major functions and capabilities of the SwipeLab software system. The functional requirements are organized by system component to provide a clear hierarchical taxonomy of the system's functionality.

3.1 Mobile Application Functionality

The mobile application is the primary interface for citizen scientists to classify images and engage with the gamification features.

3.1.1 User Authentication and Account Management

3.1.1.1 User Registration - The system shall allow new users to create an account using OAuth providers (Google, Apple ID) or email and password. The system shall verify email addresses through a confirmation link. The system shall create a unique user ID and initialize a user profile with default settings and initial neutral credibility score upon registration.

3.1.1.2 User Login - The system shall authenticate users via OAuth providers or email/password credentials. The system shall create a session token upon successful authentication that expires after 30 days of inactivity. The system shall automatically log in users if a valid session token exists locally. The system shall support biometric

authentication (fingerprint/face recognition) on supported devices. The system shall implement rate limiting after 5 failed login attempts.

3.1.1.3 User Profile Management - The system shall allow users to view and update their profile information (username, email, display name, notification preferences). The system shall allow users to change their password (for email/password accounts). The system shall allow users to delete their account and associated data (GDPR compliance).

3.1.2 Classification Functionality

3.1.2.1 Image Display and Navigation - The system shall retrieve and display images from the backend classification queue. The system shall display the classification question clearly above the image. The system shall pre-load the next 3-5 images in the background for smooth user experience. The system shall support zooming and panning on images for detailed examination. The system shall display reference pictures (if available) via an accessible info button.

3.1.2.2 Classification Input - The system shall accept classification decisions via swipe gestures (RIGHT for "Yes", LEFT for "No", UP for "I Don't Know", DOWN for "Trash"). The system shall accept classification decisions via button taps as an alternative to swipes. The system shall provide immediate visual feedback (animation) upon classification. The system shall record the response time from image display to classification decision and capture gesture metadata (velocity, direction) for quality analysis.

3.1.2.3 Classification Submission - The system shall submit classifications to the backend API immediately when online. The system shall queue up to 100 classifications locally when offline and automatically sync them when connectivity is restored. The system shall record metadata with each classification including user ID, image ID, classification decision, response time, timestamp, device metadata, and session ID.

3.1.2.4 Trap Picture Handling - The system shall present trap pictures (gold standard images) indistinguishably from regular images. The system shall not reveal which images are trap pictures to the user. The system shall optionally provide subtle feedback after trap picture classification (configurable by researcher).

3.1.3 Gamification Features

3.1.3.1 Points System - The system shall award 10 points for each Yes/No classification and 5 points for each "I Don't Know" classification. The system shall display the points earned immediately after each classification. The system shall maintain a running total of user points and sync with the backend server after each classification.

3.1.3.2 Badges and Achievements - The system shall unlock badges based on predefined criteria (classification milestones: 10, 50, 100, 500, 1000, 5000, 10000; accuracy milestones: 80%, 90%, 95% on trap pictures; streak milestones: 3, 7, 30, 100 consecutive days; speed achievements; specialist badges). The system shall display a celebration animation when a badge is unlocked. The system shall maintain a collection of all earned badges in the user profile and display progress toward next badge unlock.

3.1.3.3 Leaderboards - The system shall display leaderboards with Daily, Weekly, Monthly, and All-Time timeframes. The system shall display top 10 users for each leaderboard showing rank position, username, total classifications, and total points. The system shall highlight the current user's position regardless of rank. The system shall update leaderboards hourly for Daily, every 6 hours for Weekly. The system shall allow users to opt-out of leaderboard display.

3.1.3.4 Streaks and Daily Login Rewards - The system shall track consecutive days of user activity (classification streak). The system shall award bonus points for maintaining streaks (e.g., 2x multiplier on day 7). The system shall display current streak count prominently in the profile. The system shall send a reminder notification if a user is at risk of breaking their streak.

3.1.4 Progress and Statistics Display

3.1.4.1 Personal Statistics - The system shall display user statistics including total classifications completed, total points earned, current credibility level (visual indicator), accuracy rate on trap pictures, current streak, total contribution to research (percentage), average response time, most classified organism types, and account activity metrics.

3.1.5 Tutorial and Onboarding

3.1.5.1 First-Time User Tutorial - The system shall present an introductory tutorial to new users explaining SwipeLab's purpose, classification mechanics, gamification system, and privacy policy. The system shall provide 5 practice classifications with immediate feedback. The system shall allow users to skip the tutorial but make it accessible from settings for later review.

3.1.5.2 Task-Specific Tutorials - The system shall present a brief tutorial when a user encounters a new classification task for the first time, displaying reference pictures with annotated key features and providing 2-3 practice examples. The system shall allow users to access reference pictures anytime via an info button during classification.

3.1.6 Notifications

3.1.6.1 Push Notifications - The system shall send push notifications (if enabled by user) for badge unlocks, significant leaderboard movements, new classification tasks, streak at-risk warnings, and weekly contribution summaries. The system shall limit notifications to a maximum of 1 per day. The system shall allow users to configure notification preferences by type.

3.1.7 Offline Mode

~~**3.1.7.1 Offline Classification** - The system shall cache up to 20 images locally for offline classification. The system shall allow users to classify cached images without internet connectivity. The system shall queue offline classifications (up to 100) for later synchronization. The system shall display a clear offline mode indicator in the UI and disable features requiring real-time data when offline.~~

~~**3.1.7.2 Data Synchronization** - The system shall automatically sync queued classifications when connectivity is restored with sync progress display. The system shall handle sync conflicts gracefully and award points only after successful sync.~~

3.2 Backend API Server Functionality

The backend server manages all business logic, data persistence, and coordination between mobile clients and the research dashboard.

3.2.1 Authentication and Authorization

3.2.1.1 OAuth Integration - The system shall integrate with Google OAuth 2.0 and Apple Sign-In for user authentication. The system shall validate OAuth tokens and exchange them for JWT session tokens. The system shall store minimal user information from OAuth providers (email, name, profile picture URL).

3.2.1.2 JWT Token Management - The system shall generate JWT tokens with 30-day expiration for authenticated users including user ID, role, and permissions in the payload. The system shall validate JWT tokens on all authenticated API requests. The system shall implement token refresh mechanism and maintain a revoked token blacklist.

3.2.1.3 Role-Based Access Control - The system shall support three user roles (Volunteer, Researcher, Administrator) and enforce role-based permissions on all API endpoints. The system shall prevent privilege escalation attacks.

3.2.2 Classification Task Management

3.2.2.1 Task Distribution - The system shall distribute classification tasks to users based on task priority, user credibility score, previously classified images (avoid duplicates), and trap picture insertion frequency. The system shall ensure fair distribution across all users and randomize image order within tasks.

3.2.2.2 Trap Picture Insertion - The system shall insert trap pictures at configurable frequency (default: 1 in 15 images) with randomized placement. The system shall track which users have encountered which trap pictures and ensure each user encounters a representative sample.

3.2.2.3 Classification Recording - The system shall store all classification submissions with complete metadata (user ID, image ID, task ID, decision, response time, timestamp, session

ID, device metadata). The system shall ensure atomic database transactions and handle duplicate submissions (idempotency).

3.2.3 Credibility Scoring

3.2.3.1 Credibility Calculation - The system shall calculate user credibility scores based on accuracy on trap pictures using a weighted moving average where recent performance weighs more heavily. The system shall require a minimum of 10 trap picture encounters before credibility score stabilizes. The system shall update credibility scores immediately after each trap picture classification and normalize scores to a 0-100 scale.

3.2.3.2 Credibility Weighting in Consensus - The system shall weight user classifications by their credibility scores when calculating consensus using the formula:
$$\text{weighted_consensus} = \frac{\sum(\text{credibility_i} * \text{vote_i})}{\sum(\text{credibility_i})}$$
The system shall allow researchers to choose between weighted and simple majority consensus.

3.2.3.3 Fraud Detection - The system shall flag users with suspicious patterns (response times under 500ms, perfect accuracy on trap pictures, random-appearing patterns, bot-like timing). The system shall automatically reduce credibility scores for flagged accounts and notify administrators for manual review.

3.2.4 Consensus Calculation

3.2.4.1 Consensus Determination - The system shall calculate consensus for each image after receiving the configured minimum number of classifications (default: 3). The system shall determine the majority classification and calculate the consensus percentage. The system shall mark images as "complete" when consensus threshold is reached (default: 80%) and flag images with low consensus (<60%) for expert review.

3.2.4.2 Confidence Scoring - The system shall calculate a confidence score for each consensus label based on level of agreement, number of classifications, average credibility of participants, and variance in response times. The system shall store confidence scores with each consensus label.

3.2.5 Points and Rewards Management

3.2.5.1 Point Allocation - The system shall immediately award points upon successful classification submission. The system shall calculate and store cumulative point totals for each user. The system shall apply streak multipliers to point awards when applicable and sync point totals to mobile clients after each classification.

3.2.5.2 Badge Management - The system shall evaluate badge unlock criteria after each classification. The system shall store badge unlock timestamps in the database and notify mobile clients immediately when badges are unlocked. The system shall track progress toward next badge for display in mobile app.

3.2.5.3 Leaderboard Generation - The system shall calculate leaderboard rankings hourly for daily leaderboards and every 6 hours for weekly/monthly leaderboards. The system shall

handle ties by using timestamp of achievement (earliest wins). The system shall cache leaderboard data to reduce database load and support pagination (top 100 per page).

3.2.6 Data Export and API

3.2.6.1 Dataset Export — The system shall generate dataset exports in multiple formats (CSV, JSON, COCO format) with filtering by consensus threshold, minimum classifications, confidence score, and date range. The system shall include image filename/URL, consensus classification, confidence score, number of classifications, individual classifications (optional, anonymized), user credibility scores (optional, anonymized), and timestamps. The system shall generate provenance metadata and support incremental exports.

3.2.7 Notification Service

3.2.7.1 Push Notification Delivery - The system shall integrate with APNS (Apple Push Notification Service) for iOS and FCM (Firebase Cloud Messaging) for Android. The system shall queue notifications for batch delivery and retry failed deliveries up to 3 times. The system shall track notification delivery success rates.

3.2.7.2 Email Notifications - The system shall send transactional emails for email verification, password reset, account deletion confirmation, and large dataset export ready notifications. The system shall use HTML email templates with responsive design.

3.3 Research Dashboard Functionality

The research dashboard is a web-based interface for researchers to manage datasets, create classification tasks, and analyze results.

3.3.1 Researcher Authentication

3.3.1.1 Researcher Login - The system shall require researchers to authenticate with researcher-level credentials using email/password authentication. The system shall support optional two-factor authentication (2FA). The system shall create researcher sessions with 8-hour expiration for security.

3.3.1.2 Researcher Account Management - The system shall allow administrators to create researcher accounts. The system shall allow researchers to update profile information and change passwords. The system shall enforce strong password requirements for researcher accounts.

3.3.2 Classification Task Creation

3.3.2.1 Task Configuration - The system shall allow researchers to create classification tasks with task name, description, classification species, reference + gold standard images, recipients for the task. The system shall allow researchers to associate tasks with one or more experiments datasets.

3.3.2.2 Reference Picture Upload - The system shall allow researchers to upload 3-10 reference pictures for each task with captions explaining key features. The system shall allow upload of negative examples (similar-looking non-target species). The system shall generate a tutorial module from reference pictures.

3.3.2.3 Quality Parameters - The system shall allow researchers to configure minimum classifications per image (default: 3), consensus threshold (default: 80%), minimum user credibility required, and maximum classification time limit.

3.3.2.4 Task Activation - The system shall allow researchers to save tasks as drafts (inactive) or activate tasks to make them available to mobile users. The system shall allow researchers to pause or deactivate tasks. The system shall prevent deletion of tasks with existing classifications.

3.3.3 Trap Picture Management

3.3.3.1 Trap Picture Definition - The system shall allow researchers to designate 5-20% of images as trap pictures with expert labels for each (correct classification, confidence level, optional notes). The system shall suggest a diverse, representative sample of images for trap pictures.

3.3.4 Progress Monitoring and Analytics

3.3.4.1 Task Dashboard - The system shall display a comprehensive dashboard for each task showing overall progress, progress bar, estimated completion time, classification rate graph, average consensus percentage, active user count, and top contributors. The system shall update the dashboard in real-time (auto-refresh every 30 seconds).

3.3.4.2 Image-Level Analytics - The system shall allow researchers to view per-image statistics (consensus classification, agreement percentage, number of classifications, distribution of responses, average user credibility, average response time). The system shall allow filtering and sorting images by any metric and flag images with low consensus for expert review.

3.3.4.3 User Performance Analytics - The system shall display anonymized user statistics including total active users, user credibility distribution, classification volume by tier, and top contributors. The system shall allow researchers to analyze classification patterns.

3.3.4.4 Data Quality Reports - The system shall generate data quality reports showing inter-annotator agreement, consensus distribution, trap picture accuracy by user tier, and flagged problematic images. The system shall provide recommendations for improving data quality.

3.3.5 Data Export

3.3.5.1 Export Configuration - The system shall allow researchers to configure exports with file format (CSV, JSON, COCO, XML), inclusion criteria, consensus calculation method, data fields, and file organization preference. The system shall provide a preview of the first 10 rows before export.

3.3.5.2 Export Generation and Delivery - The system shall generate exports immediately for small files (<100MB) or process large exports in background with email notification. The system shall store export files for 7 days for re-download. The system shall log all export events for audit trail.

3.3.6 User Management

3.3.6.1 User Viewing - The system shall allow researchers to view anonymized user lists with user-level statistics (total classifications, credibility, last active). The system shall allow filtering users by various criteria.

3.3.6.2 User Moderation - The system shall allow administrators to suspend suspicious accounts, block specific users from specific tasks, and adjust user credibility scores manually. The system shall log all moderation actions for audit trail.

3.4 Administrative Functionality

System administrators have additional capabilities for system monitoring and maintenance.

3.4.1 System Monitoring

3.4.1.1 Health Dashboard - The system shall display real-time system health metrics (server status, active users, API response times, error rates, database performance, storage usage). The system shall alert administrators when metrics exceed thresholds.

3.4.1.2 Logging and Audit Trail - The system shall log all significant system events (user registrations/logins, classification submissions, dataset uploads, task creations/modifications, data exports, administrative actions). The system shall retain logs for 90 days.

3.4.2 User Administration

3.4.2.1 Account Management - The system shall allow administrators to create, suspend, or delete user accounts. The system shall allow administrators to change user roles and reset user passwords.

3.4.2.2 Fraud Investigation - The system shall provide administrators with tools to investigate flagged accounts (detailed classification history, response time analysis, accuracy patterns, device and IP information). The system shall allow administrators to retroactively adjust classifications from fraudulent accounts.

3.4.3 System Configuration

3.4.3.1 Parameter Tuning - The system shall allow administrators to configure global parameters (default consensus threshold, minimum classifications per image, trap picture insertion frequency, point values, badge unlock criteria, rate limits). The system shall validate parameter changes before applying.

3.4.3.2 Feature Flags - The system shall support feature flags for gradual rollout of new features. The system shall allow administrators to enable/disable features per user segment.

4. Non-Functional Requirements

4.1 Implementation Constraints

4.1.1 Speed, Capacity & Throughput

- When a user performs a swipe action (left or right), the classification should be recorded and the next image should load within 500 milliseconds.
- The app should handle image loading in the background, pre-loading the next 3-5 images to ensure smooth user experience.
- 95% of all API calls to submit classification data should complete within 2 seconds.
- Initial app launch time should not exceed 3 seconds on devices meeting minimum hardware requirements.
- The app should support a minimum of 50 classification actions per session without performance degradation.

4.1.2 Reliability & Stability

Our system reliability will be affected by the following factors:

- Quality and availability of the customer's insect image dataset API.
- Stability of the network connection between the mobile device and the backend server.

- Quality and consistency of the insect images provided in the dataset.

Reliability measures:

- The app must handle network failures gracefully by queuing classifications locally and synchronizing when connection is restored.
- If the API connection fails, the app should retry up to 3 times with exponential backoff before notifying the user.
- Local data storage must prevent data loss, ensuring that no user classifications are lost even in case of app crash.

4.1.3 Safety & Security

- All user authentication data must be transmitted using HTTPS/TLS encryption.
- User passwords must be hashed using industry-standard algorithms (e.g., bcrypt) and never stored in plain text.
- API communication between the mobile app and backend server must use secure protocols (HTTPS).
- User authentication tokens should expire after 20 days of inactivity, requiring re-authentication.
- The app will implement basic protection against SQL injection and other common security vulnerabilities.
- User classification data should be anonymized when transmitted to researchers, protecting user privacy.

4.1.4 Portability

- The application is designed for mobile devices using React Native framework.
- The app must support Android operating system version 8.0 (API level 26) or later.
- The app must support iOS operating system version 13.0 or later.
- User interface will be written in English initially, with architecture supporting future localization to other languages.
- The app must adapt to different screen sizes, supporting devices from 4.7" to 6.7" displays.
- Only portrait orientation must be supported.

4.1.5 Usability

- The app is designed for non-expert users (citizen scientists) with no prior entomological knowledge required.
- Users should be able to complete their first classification within 2 minutes of first launching the app, including onboarding tutorial.
- The swipe interface must be intuitive – 90% of first-time users should understand how to classify insects without additional instructions beyond the initial tutorial.
- The gamification elements (points, badges, progress tracking) should be visible and easy to understand.
- The app should provide clear visual feedback for each swipe action (animation, sound, or haptic feedback).
- Error messages should be clear and actionable, suggesting solutions when possible.
- The app should maintain consistent UI/UX patterns throughout all screens.

4.1.6 Availability

- The backend server should maintain 99% uptime during active research periods.
- The app should function in offline mode for core classification functionality, with automatic synchronization when internet connection is restored.
- The app should handle temporary API unavailability by informing users and allowing continued offline work.

4.2 Platform Constraints

- The development framework is React Native, chosen for cross-platform mobile development (iOS and Android).
- The backend server must integrate with the customer's existing insect image dataset API.
- The app must comply with Google Play Store submission guidelines and requirements.
- Development will use modern JavaScript (ES6+) and React Native best practices.
- The system shall provide a comprehensive RESTful API with endpoints for authentication, classification tasks, user profile, leaderboards, and statistics. The system shall return JSON responses with proper HTTP status codes. The system

shall implement API versioning in URL path and enforce rate limiting (100 requests/minute per user).

- Data visualization and reporting may require integration with the customer's existing research data systems.

4.2.1 SE Project Constraints

- Our program is interactive, and inputs come from real users performing swipe classifications on insect images.
- The device will not be simulated – the app will be demonstrated on actual mobile devices (Android and/or iOS smartphones).
- We will use samples of actual insect images provided by the customer for testing and demonstration purposes.
- The system requires access to the customer's insect image dataset API for retrieving images to classify.
- Development will begin with a local development server and test dataset, transitioning to the production API in later stages.
- Testing will be conducted on multiple device types to ensure compatibility across different screen sizes and OS versions.
- Pilot user evaluation with researchers will be conducted in Week 12 to gather feedback and validate the system.
- Final presentation and demo will showcase the app running on physical devices with live classification functionality, demonstrating both the user interface and the gamification features.
- The project timeline is 13 weeks, requiring efficient development iterations to meet all milestones.

4.3 Special restrictions & limitations

- The system depends on the availability and accessibility of the customer's insect image dataset and API.
- Classification accuracy and research value depend on the quality of images provided by the researchers.
- Gamification mechanics must be designed carefully to maintain scientific validity and not bias user classifications.
- The app requires active internet connection for initial image download and periodic synchronization, though core functionality will work offline.

- The number and variety of insect species available for classification will be limited to what is provided in the taxonomy of StarDBI.
- User engagement and retention depend on effective gamification design and regular content updates (new images and challenges).

5. Risk assessment & Plan for the proof of concept

5.1 Technical and Project Risks

The following are the primary risks identified for the project:

1. API Integration with STARdbi

The system relies on retrieving sub-images and metadata from the STARdbi database.

Risk: API may not support required data formats, batching, pagination, or real-time retrieval.

Impact: Core functionality blocked; no labeling tasks available.

2. Dataset Scale and Image Throughput

The dataset contains ~350,000 images, growing continuously.

Risk: Slow loading times or bottlenecks when fetching images for swiping SINCE WE ARE NOT THE ONLY USERS OF THIS STARDBI API (there are other projects that use this server).

Impact: Poor user experience; abandoned tasks; unusable platform.

3. Anti-Bot & Quality Assurance Challenges

Users may try to cheat for points or automated bots may attempt classifications.
Risk: Label corruption and unreliable training data.
Impact: Scientific validity and ML training compromised.

4. Gamification & Engagement Risk

The success of the platform depends heavily on long-term user engagement.
Risk: Insufficient motivation → low participation → dataset not labeled.
Impact: Project fails to meet scientific goals.

5. Technology Learning Curve

The team must integrate multiple technologies: React Native (Expo), Spring Boot or Node backend, PostgreSQL, Google Authentication, image serving optimization.
Risk: Development delays or architectural mistakes due to unfamiliarity.
Impact: Timeline slippage; poor system reliability.

6. Device & Platform Variability

Users will access via mobile/desktop, different OS versions and screen sizes.
Risk: UI inconsistencies, unexpected bugs, accessibility issues.
Impact: Reduced participation and trust.

7. Security & Authentication

User accounts and progress must be protected.
Risk: Unauthorized API access or data leakage.
Impact: Privacy issues, misuse of system resources.

5.2 Proof-of-Concept Goals

The purpose of the proof-of-concept is to validate the foundational components of SwipeLab before full implementation. The PoC must provide evidence that the system can:

- ✓ Integrate with the STARdbi dataset API
- ✓ Load and display images efficiently for swipe-based classification

- ✓ Store classifications and user activity reliably
- ✓ Handle basic authentication flows
- ✓ Demonstrate initial gamification mechanics (e.g., points)
- ✓ Prevent basic bot-like behavior
- ✓ Validate usability and interaction flow (swipe speed, clarity, accessibility)

The PoC is not meant to include full gamification, full scoring logic, or full UX polish. Its purpose is to reduce project risk early.

5.3 Scope of the PoC Prototype

The PoC will include the following limited but crucial components:

1. Minimal Mobile Interface

- A simple React Native screen that displays one insect image at a time.
- Basic swipe-left / swipe-right interaction.
- Simple buttons as fallback for accessibility.

2. Basic API Retrieval Test

- Integration with the STARdbi API using a mock or real endpoint.
- Fetching a small batch of images (10–20).
- Logging and verifying metadata (trap ID, Task ID, timestamp).

3. Local or Lightweight Backend

- A basic Spring Boot or Node server that:
 - Receives classification events
 - Writes them to a temporary PostgreSQL table

- Performs basic validation and timestamping

4. Basic Authentication Flow

- Google OAuth login
- Store minimal user profile (UID + nickname)

5. Early Gamification Logic

- Incremental point system: +1 for each classification
- Show the user their current score
- No leaderboards yet — only proof the scoring flow works

6. Early Anti-Bot Check

- Measure swipe speed
- If speed is “non-human fast”, require a simple CAPTCHA popup
- Logging user behavior for later analysis

7. Minimal Consensus Simulation

- Aggregate labels (Yes/No) from multiple users
- Calculate majority vote
- Display results on admin console

5.4 How the PoC Reduces Project Risk

The PoC directly mitigates the major risks identified earlier by providing early validation in the following areas:

Risk Category	How the PoC Mitigates It
---------------	--------------------------

API integration	Ensures endpoint compatibility, metadata correctness, and image retrieval feasibility.
Performance and throughput	Measures loading time, caching needs, and image handling on real devices.
Bot detection	Validates whether timing-based heuristics or verification prompts are effective.
User engagement	Tests swipe UX and checks whether users understand the task without explanation.
Technology learning	Allows team to practice React Native + backend + PostgreSQL before building the full system.
Data quality	Tests small-scale consensus logic before expanding to full labeling pipeline.
Security	Validates authentication flow and prevents unauthorized access, even at early stage.

5.5 Expected Outcomes of the PoC

The PoC should provide:

- Better understanding of requirements:
Real images, real user behavior, real performance metrics.
- Understanding of technology limitations:
Identifies bottlenecks in image rendering, API speed, database writes, etc.
- Assessment of early design decisions:
Helps validate architecture and avoid expensive redesign later.

- Visualization of the final system:
Stakeholders can see and interact with a working mini-version.
- Risk reduction:
Confirms that the project is feasible and that the team can successfully implement the larger system.

Appendices

This appendix includes supplementary information that supports the understanding and validation of the requirements in this document. It provides technical details, background information, domain definitions, and references that may assist stakeholders, developers, and reviewers throughout the project lifecycle.

Appendix A — I/O Formats and Data Structures

This section describes the expected formats for data exchanged between SwipeLab and external systems (e.g., STARdbi API), as well as internal data structures.

A.1 Image Input Format (from STARdbi)

Data Source: STARdbi trap-scan database

Each classification task uses *sub-images* extracted from larger trap scans.

Typical API Response Structure (example):

```
{
  "image_id": "stardbi_314159",
  "trap_id": "T48",
  "site": "negev_03",
  "timestamp": "2023-05-14T21:33:00Z",
  "image_url": "https://.../image_314159.jpg",
  "metadata": {
    "trap_scan_id": "scan_102",
    "x": 341,
```

```
    "y": 928,  
    "width": 64,  
    "height": 64  
  }  
}
```

Notes:

- Images are served as JPEG/PNG.
- Some metadata fields may change depending on the final API design with the client.
- The prototype will initially use a simplified mocked structure.

A.2 User Classification Output Format

When a user submits a swipe classification:

```
{  
  "user_id": "user_123",  
  "image_id": "stardbi_314159",  
  "task_type": "wasp_binary",  
  "label": "positive",  
  "timestamp": 1711380010,  
  "client_device": "android",  
  "swipe_duration_ms": 892  
}
```

A.3 Aggregated Label Output Format (for AI training)

Produced after consensus:

```
{  
  "image_id": "stardbi_314159",  
  "task_type": "wasp_binary",  
  "consensus_label": "positive",  
  "confidence": 0.86,  
  "votes_positive": 18,  
  "votes_negative": 3,  
  "num_users": 21  
}
```

}

Appendix B — Background Information

This section includes context about the domain of the project and the underlying scientific need.

B.1 The STARdbi Initiative

SwipeLab is part of the larger **STARdbi** project, a database and research infrastructure for insect-trap imagery collected from 52 ecological sites.

Key Statistics:

- **1740 trap scans**
- **350,000+ insect images**
- Used by researchers in:
 - Ecology
 - Agriculture
 - Public health
- Purpose: Identify insect populations, species distribution, and ecological trends.

B.2 The Scientific Challenge

Research requires *labeled images*, but:

- Expert labeling is slow
- It is very expensive
- It is a major bottleneck preventing further analysis
- AI-based detection/classification models require large labeled datasets

SwipeLab converts a complex task into a **binary, simple, gamified micro-task** that non-experts can perform.

B.3 Similar Projects / Market Survey

The SwipeLab platform relates to existing “citizen science” projects:

- **Zooniverse** – large-scale crowdsourced scientific tasks
- **Foldit** – gamified scientific contribution
- **iNaturalist** – crowd-supported species classification

However, SwipeLab is unique in:

- Its binary swipe interaction (Tinder-style)
 - Its focus on micro-classifications
 - Its scientific integration with STARdbi
 - Its focus on training AI classifiers directly
-

Appendix C — Survey & User Research

If user studies or interviews are conducted later, they can be documented here.
For now, planned forms of user study include:

- Short usability test with 5–10 non-expert users
 - Feedback on clarity of binary task
 - Assessment of swipe UX and image visibility
 - Engagement-motivation survey (points, streaks, badges)
 - Prototype performance testing on multiple devices
-

Appendix D — Glossary

This glossary defines domain-specific terms used throughout the document.

Binary Classification

A task where an item is categorized into one of two categories (e.g., “Wasp / Not Wasp”).

Consensus Labeling

Aggregating multiple user labels to determine a final classification with higher confidence.

Citizen Science

A scientific approach in which non-experts contribute data or analysis to help researchers.

Entomology

The scientific study of insects.

Gamification

The use of game-like mechanics (points, badges, leaderboards) to increase user engagement.

Insect Trap Scan

A large, high-resolution image produced by scanning an insect trap containing many captured insects.

STARdbi

A BGU research project providing a structured database of insect trap images for ecological, agricultural, and health-related studies.

Swipe Interaction

A gesture commonly used in mobile apps (left/right swiping) to classify or navigate.

Trap Site

A physical location where insect traps are placed to collect ecological data.

Training Dataset

A collection of labeled images used to train machine learning models.

Throughput

The number of images that can be loaded and classified per unit of time.

PoC (Proof of Concept)

A small experiment or prototype aimed at validating feasibility or identifying risks.

Appendix E — Potential Extensions (Future Work)

Future enhancements that may improve SwipeLab or extend its capabilities:

- Multi-class classification (instead of binary)
 - Image segmentation to isolate insects automatically
 - User reputation score influencing label weight
 - Adaptive task difficulty
 - Integration with trained ML models (human-AI collaboration)
 - Global competitions and public achievements
 - Expert dashboards for reviewing low-confidence labels
 - Offline mode with delayed sync
-

Appendix F — References

- STARdbi official site: <https://stardbi.cs.bgu.ac.il/home/welcome>
- Industry best practices on citizen science platforms
- Tinder-style UX pattern references

